

ships with their software. First, we would need to declare the variables. To save space, we'll just show one example:

```
var 1..9: S;
```

Here 1..9 specifies the range of values S can take. Next, we define the constraint that all values must be different:

```
constraint alldifferent([S,E,N,D,M,O,R,Y]);
```

Finally, the biggest constraint we have is of the equation that must be satisfied. We can represent that as:

```
constraint 1000 * S + 100 * E + 10 * N + D
+ 1000 * M + 100 * O + 10 * R + E
= 10000 * M + 1000 * O + 100 * N + 10 * E + Y;
```

After this, all we have to do is instruct the program to solve the constraints as specified. This is accomplished with a simple statement:

```
solve satisfy;
```

That solves the problem for us. This was a rather simple problem that you can solve by hand. So let's look at something a lot tougher. This one is called the n-Queens problem. It requires one to place n queens on an n-by-n chess board such that no queen can attack any other queen. If this sounds a little confusing, don't worry, we will look at this problem in detail.

Solving a Constraint Program by Hand:

In chess, the queen has the widest range of motion. It can move any number of spaces horizontally, vertically, and diagonally in any direction. So placing them such that no queen can attack any other one requires quite a bit of strategy. Given this problem, we need to figure out what should be the variables, and what should be the constraints.

Before we dive into solving this using a computer, let's take a simple version of this problem and see how one could solve it by hand. We'll look at a 4×4 problem since there is no solution possible for 2×2 and 3×3 boards — think about it and you'll see why.

First of all, we can immediately see a constraint. No two queens can be on the same row, or the same column. So we'll take our 4 queens, and assign each its own column. Now we just need to figure out which row each should be on.

Let's start by putting the first queen in the first possible location, the top-left corner. This eliminates all spaces on the same row, column and diagonal. For the second queen, we have two options on the second column so let's put it in the first one for now and move on. The second we do this, we realise that this won't work. We have eliminated all spaces in the third column.

At this point, we need to backtrack. We now know that if the first queen is where it is, the second one can't be in this location. Let's move

*pointers

>>Interesting developer videos



*Discrete Optimization at Coursera

>>Coursera's discrete optimization course has a module dedicated to constraint programming among other interesting topics.

<http://dgit.in/CourOptim>



*Intro to Constraint Programming

>>This two-part video series titled "An Introduction to Constraint Programming and Combinatorial Optimisation" does just what it says.

<http://dgit.in/ConstraintPrg>



*Prolog Programming

>>This video series introduces you to Prolog, a logic programming language that supports constraint programming natively.

<http://dgit.in/PrologProg>



*Java Cream Sample App

>>Cream is a constraint programming library for Java. This short video should help you get started programming with it.

<http://dgit.in/JAVACream>



it to the next possible location at row 4. And, let's stop right here. We can see two free spaces, and both are diagonal to each other. This will never work. What have we gotten ourselves into? Right?!

We have eliminated both possible locations for the second queen, from this we can conclude that the first queen is misplaced. At this point, we need to go back two steps, and move the first queen to another location, the second row.

On doing this, we immediately see that there is only one spot in the second column, so let's place one there as well. This in turn leaves only one spot for the third queen in the first row. This now leaves one space for our final queen. And so ladies and gentlemen, we finally have a solution.

Now let's do this for 10 queens. Just kidding! Let's look at a possible constraint programming solution that works for any number of queens. The program we will build below can give you a solution for as many as 30 queens in under a second.

Building a Constraint Program:

At first glance, you might think that each queen will need two variables, a row, and a column. Certainly it is possible to develop a solution that way. We have already discovered above though, that each queen will be in a different column. So why not just assign one to each? Queen 1 in column 1, and so on. We can even use an array for this where the index can be the column number and the row can be the value of the array at that index.

The first constraint we can place now is that all the queens are in different rows. We have done this before when we wanted all letters in the "SEND+MORE=MONEY" problem to be distinct, we use the alldifferent command.

Next comes the tricky part, how do we convey that no two queens should be along the same diagonal? A few smart people have discovered the trick to this, and it is quite clever when you see how it works.

The solution is to use two constraints, the first constraint is that no two queens can have the value of row number + column number the same; and the second constraint is that no two queens can have their row number - column number of the same value.

This might seem counter-intuitive to you, so we have attached an illustration that shows a 4×4 matrix where each cell has the sum row + column number in it; and another matrix that shows the difference row - column number. You'll see that two cells having the same value corresponds to them being along the same diagonal.